

KONGU ENGINEERING COLLEGE

IOT BASED AUTOMATED COLOR SORTING SYSTEM USING ESP8266

EXCERSICE - 09

HEMAVARSHINI S – 24MER017

KAVIN KUMARAN S – 24MER023

KOWSIKA K – 24MER026

VISHNUPRASATH B – 24MER062



KONGU ENGINEERING COLLEGE

— TRANSFORM YOURSELF —

IoT Based Automated Color Sorting System Using ESP8266

1.INTRODUCTION

The Automated Color Sorting System is designed to automatically detect and sort objects based on their color. The system uses an ESP8266 NodeMCU as the main controller, a TCS3200 color sensor to identify the color of the object, an IR sensor to detect the presence of an object, and a servo motor to direct the object into the appropriate sorting section.

The detected color and sensor information are transmitted through Wi-Fi to ThingSpeak Cloud for monitoring and data visualization.

The system can identify three basic colors:

- Red
- Green
- Blue

The project demonstrates the integration of embedded systems, sensors, automation, servo control, IoT and cloud monitoring.

2. PROBLEM STATEMENT

In conventional material-sorting systems, objects are often sorted manually or using expensive industrial automation equipment.

Manual sorting has several limitations:

- It requires human effort.
- Sorting speed is limited.
- Human errors may occur.
- Continuous monitoring is difficult.
- Data regarding the sorting process is generally not stored.
- Industrial automated systems can be expensive for small-scale applications.

Therefore, there is a need for a low-cost automated color-sorting system that can detect an object's color, automatically sort it and monitor the sorting data remotely.

3. PROPOSED SOLUTION

The proposed system uses an ESP8266 NodeMCU to control the entire sorting operation.

An IR sensor detects when an object is placed in the sorting area. Once an object is detected, the TCS3200 color sensor measures the red, green and blue components of the object.

The ESP8266 compares the sensor readings with calibrated threshold values and determines the color.

According to the detected color:

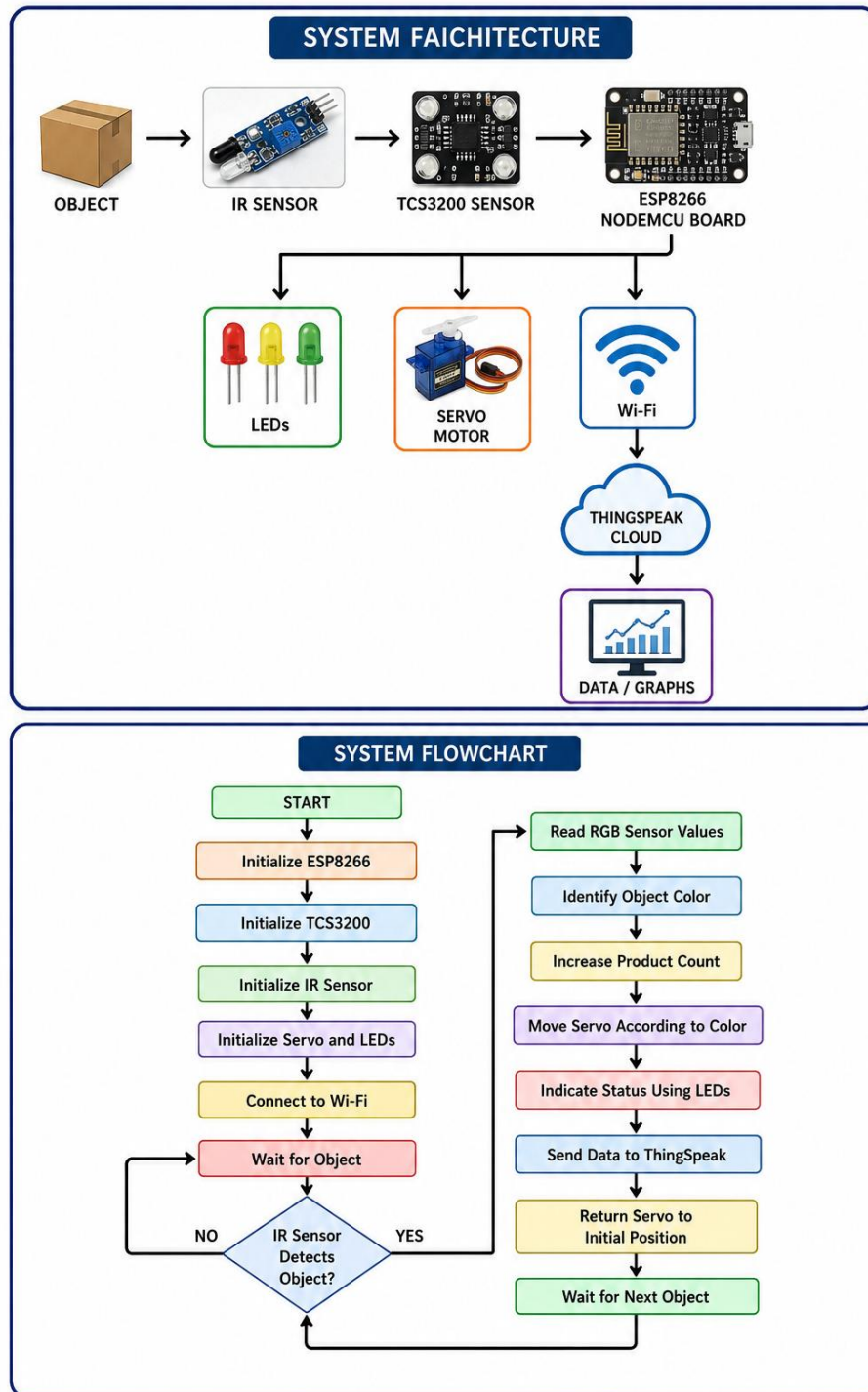
- The servo motor rotates to the corresponding position.
- The object is directed to the appropriate sorting section.
- LEDs indicate the sorting status.
- The detected color, sensor values, product count and servo angle are uploaded to ThingSpeak.

Main Features

1. Automatic object detection.
2. Automatic color identification.
3. Servo-based sorting.
4. Real-time sensor monitoring.
5. Product counting.
6. Wi-Fi connectivity.
7. ThingSpeak cloud monitoring.
8. Low-cost implementation.

4. SYSTEM ARCHITECTURE

The overall architecture of the system is:



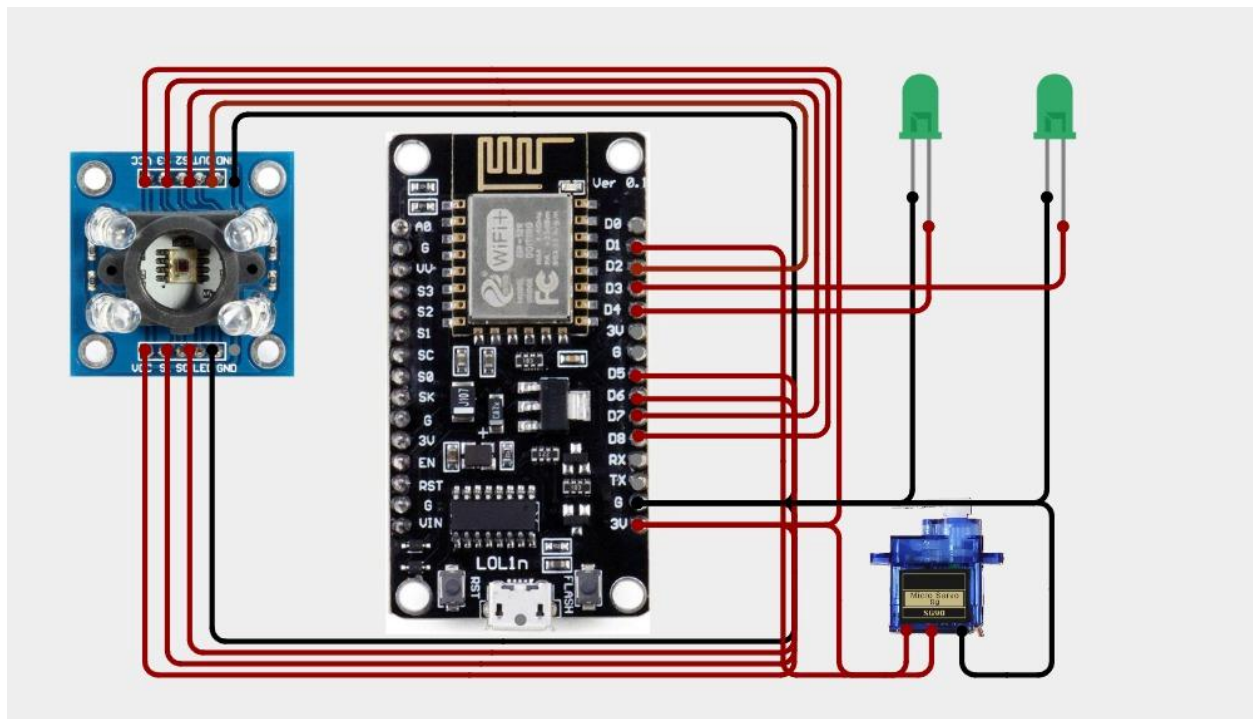
5. CIRCUIT TABLE

Component	ESP8266 NodeMCU Pin	Purpose
TCS3200 S0	D1	Frequency scaling
TCS3200 S1	D2	Frequency scaling
TCS3200 S2	D5	Color filter selection
TCS3200 S3	D6	Color filter selection
TCS3200 OUT	D7	Frequency output
IR Sensor OUT	D3	Object detection
Servo Signal	D4	Servo control
Red LED	D0	Red indication
Green LED	D8	Green indication
TCS3200 VCC	3.3V	Power
TCS3200 GND	GND	Ground
IR VCC	3.3V	Power
IR GND	GND	Ground
Servo VCC	External 5V	Servo power
Servo GND	GND	Common ground

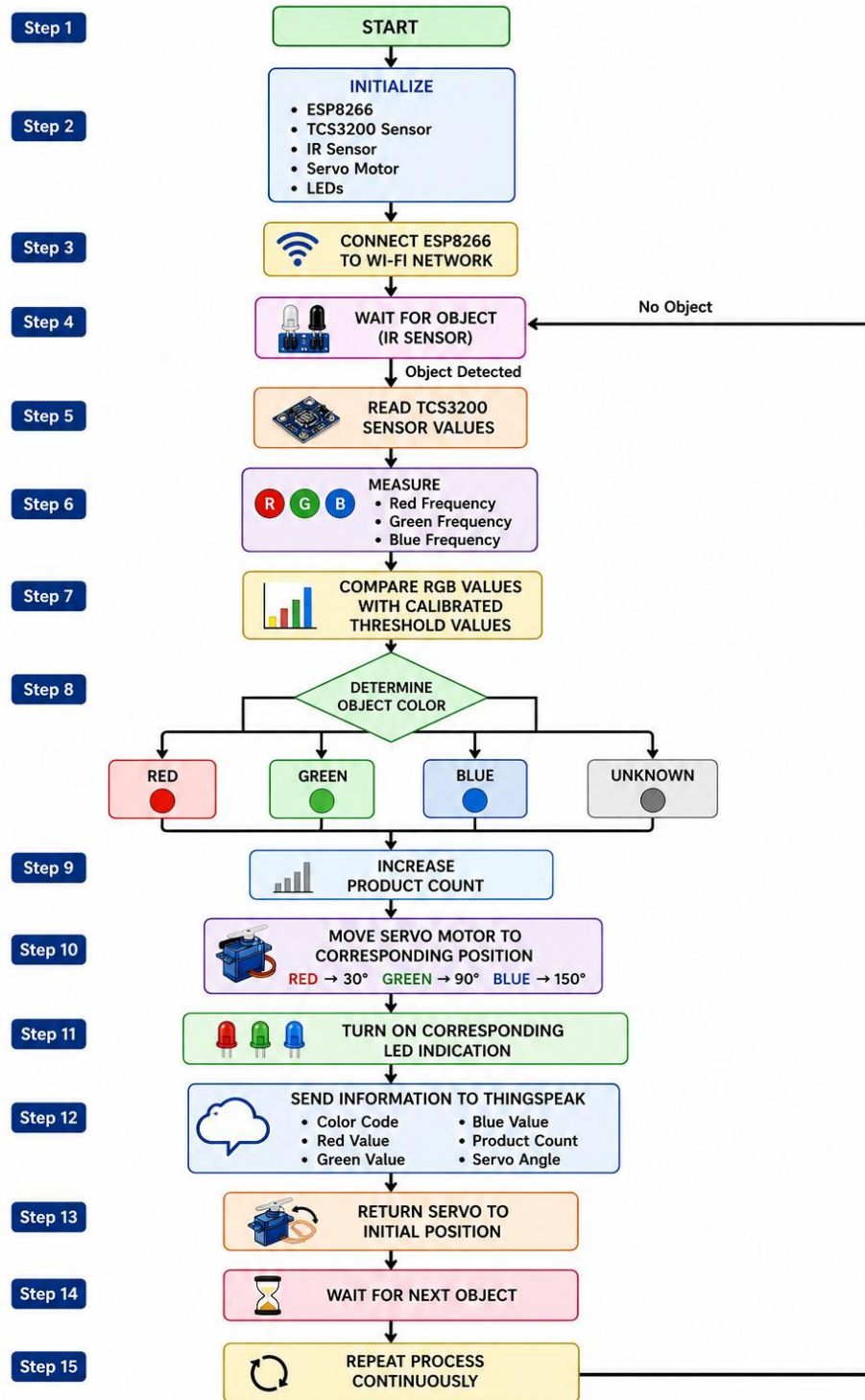
6. CIRCUIT DESIGN

Components Required

- ESP8266 NodeMCU
- TCS3200 / TCS230 Color Sensor
- IR Proximity Sensor
- Servo Motor
- Red LED
- Green LED
- 220 Ω resistors – 2
- Breadboard
- Jumper wires
- External 5V supply for servo
- USB cable
- Wi-Fi connection



7. ALGORITHM



8. CODING

```
#include <ESP8266WiFi.h>

#include <ESP8266HTTPClient.h>

#include <Servo.h>

const char* ssid = "YOUR_WIFI_NAME";

const char* password = "YOUR_WIFI_PASSWORD";

const char* server = "http://api.thingspeak.com/update";

String writeAPIKey = "YOUR_THINGSPEAK_WRITE_API_KEY";


#define S0 D1

#define S1 D2

#define S2 D5

#define S3 D6

#define SENSOR_OUT D7


#define IR_SENSOR D3

#define SERVO_PIN D4


#define RED_LED D0

#define GREEN_LED D8


Servo sortingServo;

int redValue;
```



```
int greenValue;
```

```
int blueValue;
```

```
int productCount = 0;
```

```
String detectedColor = "UNKNOWN";
```

```
int servoAngle = 90;
```

```
void setup()
```

```
{
```

```
Serial.begin(115200);
```

```
// TCS3200
```

```
pinMode(S0, OUTPUT);
```

```
pinMode(S1, OUTPUT);
```

```
pinMode(S2, OUTPUT);
```

```
pinMode(S3, OUTPUT);
```

```
pinMode(SENSOR_OUT, INPUT);
```

```
// IR sensor
```

```
pinMode(IR_SENSOR, INPUT);
```

```
// LEDs
```

```
pinMode(RED_LED, OUTPUT);
```

```
pinMode(GREEN_LED, OUTPUT);
```

```
// Frequency scaling = 20%  
  
digitalWrite(S0, HIGH);  
digitalWrite(S1, LOW);  
  
// Servo  
sortingServo.attach(SERVO_PIN);  
sortingServo.write(90);  
  
digitalWrite(RED_LED, LOW);  
digitalWrite(GREEN_LED, LOW);  
  
// Wi-Fi  
WiFi.begin(ssid, password);  
  
Serial.println();  
Serial.print("Connecting to Wi-Fi");  
  
while (WiFi.status() != WL_CONNECTED)  
{  
    delay(500);  
    Serial.print(".");  
}  
  
Serial.println();  
Serial.println("Wi-Fi Connected!");  
Serial.print("IP Address: ");  
Serial.println(WiFi.localIP());
```

```
Serial.println("=====");  
Serial.println("COLOR SORTING SYSTEM READY");  
Serial.println("=====");  
}  
  
int readRed()  
{  
    digitalWrite(S2, LOW);  
    digitalWrite(S3, LOW);  
  
    delay(50);  
  
    return pulseIn(SENSOR_OUT, LOW, 100000);  
}  
  
int readGreen()  
{  
    digitalWrite(S2, HIGH);  
    digitalWrite(S3, HIGH);  
  
    delay(50);  
  
    return pulseIn(SENSOR_OUT, LOW, 100000);  
}  
  
int readBlue()  
{  
    digitalWrite(S2, LOW);
```

```
digitalWrite(S3, HIGH);
```

```
delay(50);
```

```
return pulseIn(SENSOR_OUT, LOW, 100000);
```

```
}
```

```
String detectColor()
```

```
{
```

```
redValue = readRed();
```

```
greenValue = readGreen();
```

```
blueValue = readBlue();
```

```
Serial.println();
```

```
Serial.println("----- SENSOR VALUES -----");
```

```
Serial.print("RED  : ");
```

```
Serial.println(redValue);
```

```
Serial.print("GREEN : ");
```

```
Serial.println(greenValue);
```

```
Serial.print("BLUE  : ");
```

```
Serial.println(blueValue);
```

```
if (redValue < greenValue * 0.75 &&
```

```
redValue < blueValue * 0.75)
```

```
{
```

```
return "RED";
```

```
}
```

```
else if (greenValue < redValue * 0.75 &&  
greenValue < blueValue * 0.75)
```

```
{
```

```
return "GREEN";
```

```
}
```

```
else if (blueValue < redValue * 0.75 &&  
blueValue < greenValue * 0.75)
```

```
{
```

```
return "BLUE";
```

```
}
```

```
else
```

```
{
```

```
return "UNKNOWN";
```

```
}
```

```
}
```

```
void sortObject(String color)
```

```
{
```

```
digitalWrite(RED_LED, LOW);
```

```
digitalWrite(GREEN_LED, LOW);
```

```
if (color == "RED")
```

```
{  
Serial.println("Sorting RED object...");  
  
digitalWrite(RED_LED, HIGH);  
  
servoAngle = 30;  
sortingServo.write(servoAngle);  
  
delay(1000);  
}  
  
else if (color == "GREEN")  
{  
Serial.println("Sorting GREEN object...");  
  
digitalWrite(GREEN_LED, HIGH);  
  
servoAngle = 90;  
sortingServo.write(servoAngle);  
  
delay(1000);  
}  
  
else if (color == "BLUE")  
{  
Serial.println("Sorting BLUE object...");
```

```
servoAngle = 150;
sortingServo.write(servoAngle);

delay(1000);
}

else
{
Serial.println("Unknown object!");

servoAngle = 90;
sortingServo.write(servoAngle);

delay(500);
}

digitalWrite(RED_LED, LOW);
digitalWrite(GREEN_LED, LOW);

// Return servo to center
sortingServo.write(90);

delay(500);
}

void sendToThingSpeak()
{
```

```
if (WiFi.status() != WL_CONNECTED)
{
  Serial.println("Wi-Fi disconnected!");
  return;
}
```

```
WiFiClient client;
HTTPClient http;
```

```
String url = String(server);
```

```
url += "?api_key=" + writeAPIKey;
```

```
// Field 1 = Color Code
```

```
int colorCode = 0;
```

```
if (detectedColor == "RED")
```

```
colorCode = 1;
```

```
else if (detectedColor == "GREEN")
```

```
colorCode = 2;
```

```
else if (detectedColor == "BLUE")
```

```
colorCode = 3;
```

```
// Field values
```

```
url += "&field1=" + String(colorCode);
```



```
url += "&field2=" + String(redValue);
url += "&field3=" + String(greenValue);
url += "&field4=" + String(blueValue);
url += "&field5=" + String(productCount);
url += "&field6=" + String(servoAngle);

Serial.println();

Serial.println("Sending data to ThingSpeak...");

http.begin(client, url);

int httpCode = http.GET();

if (httpCode > 0)
{
    Serial.print("ThingSpeak Response: ");
    Serial.println(httpCode);

    String response = http.getString();

    Serial.print("Entry ID: ");
    Serial.println(response);
}

else
{
    Serial.println("Error sending data!");
}
```

```
}
```

```
http.end();
```

```
}
```

```
void loop()
```

```
{
```

```
int objectDetected = digitalRead(IR_SENSOR);
```

```
// Depending on IR sensor module,
```

```
// LOW may indicate object detection.
```

```
if (objectDetected == LOW)
```

```
{
```

```
Serial.println();
```

```
Serial.println("=====");
```

```
Serial.println("OBJECT DETECTED");
```

```
Serial.println("=====");
```

```
delay(300);
```

```
detectedColor = detectColor();
```

```
Serial.print("Detected Color: ");
```

```
Serial.println(detectedColor);
```

```
productCount++;

Serial.print("Product Count: ");
Serial.println(productCount);

sortObject(detectedColor);

sendToThingSpeak();

Serial.println("=====");
Serial.println("SORTING COMPLETED");
Serial.println("=====");

delay(1500);
}

delay(100);
}
```

9. SYSTEM WORKING

The working of the system takes place in the following sequence.

1. Object Detection

The IR sensor continuously monitors the sorting area.

When an object comes in front of the IR sensor, the sensor sends a signal to the ESP8266.

2. Color Detection

The ESP8266 activates the TCS3200 sensor and reads the red, green and blue components.

The TCS3200 produces a frequency output corresponding to the intensity of the selected color.

3. Color Identification

The ESP8266 compares the measured values.

The color with the strongest corresponding response is selected as the detected color.

4. Servo Operation

After detecting the color, the ESP8266 rotates the servo motor.

Example:

RED → 30°

GREEN → 90°

BLUE → 150°

The servo physically directs the object toward the appropriate collection section.

5. LED Indication

The LEDs provide a visual indication of the detected/sorting condition.

6. Product Counting

Every detected object increases the product count by one.

7. IoT Monitoring

The ESP8266 connects to Wi-Fi and uploads the following information to ThingSpeak:

Color

RGB Sensor Values

Product Count

Servo Angle

8. Cloud Visualization

ThingSpeak stores the information and provides graphs that can be used to monitor the sorting process.

10. BILL OF MATERIALS

S.No	Component	Quantity	Purpose
1	ESP8266 NodeMCU	1	Main controller and Wi-Fi
2	TCS3200 Color Sensor	1	Color detection
3	IR Proximity Sensor	1	Object detection
4	Servo Motor	1	Object sorting
5	Red LED	1	Status indication
6	Green LED	1	Status indication
7	220 Ω Resistor	2	LED current limiting
8	Breadboard	1	Circuit assembly
9	Jumper Wires	As required	Connections
10	5V Power Supply	1	Servo power
11	USB Cable	1	ESP8266 programming
12	Wi-Fi Network	1	IoT communication

11. PROTOTYPE IMPLEMENTATION

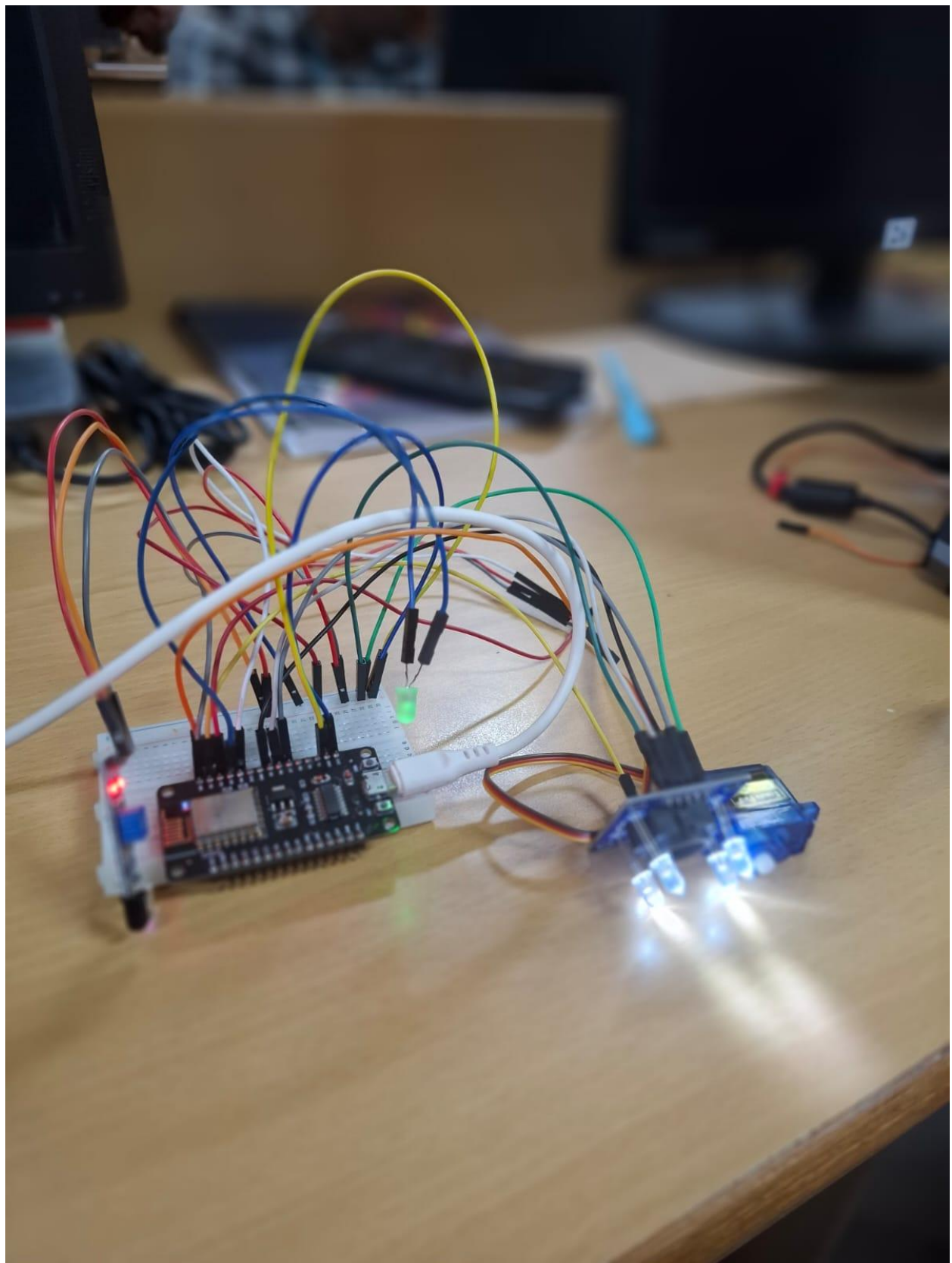
The prototype consists of the ESP8266 NodeMCU connected to the TCS3200 color sensor, IR sensor, servo motor and LEDs.

The object is placed on the sorting platform.

The IR sensor first detects the presence of the object. The ESP8266 then reads the color information from the TCS3200 sensor.

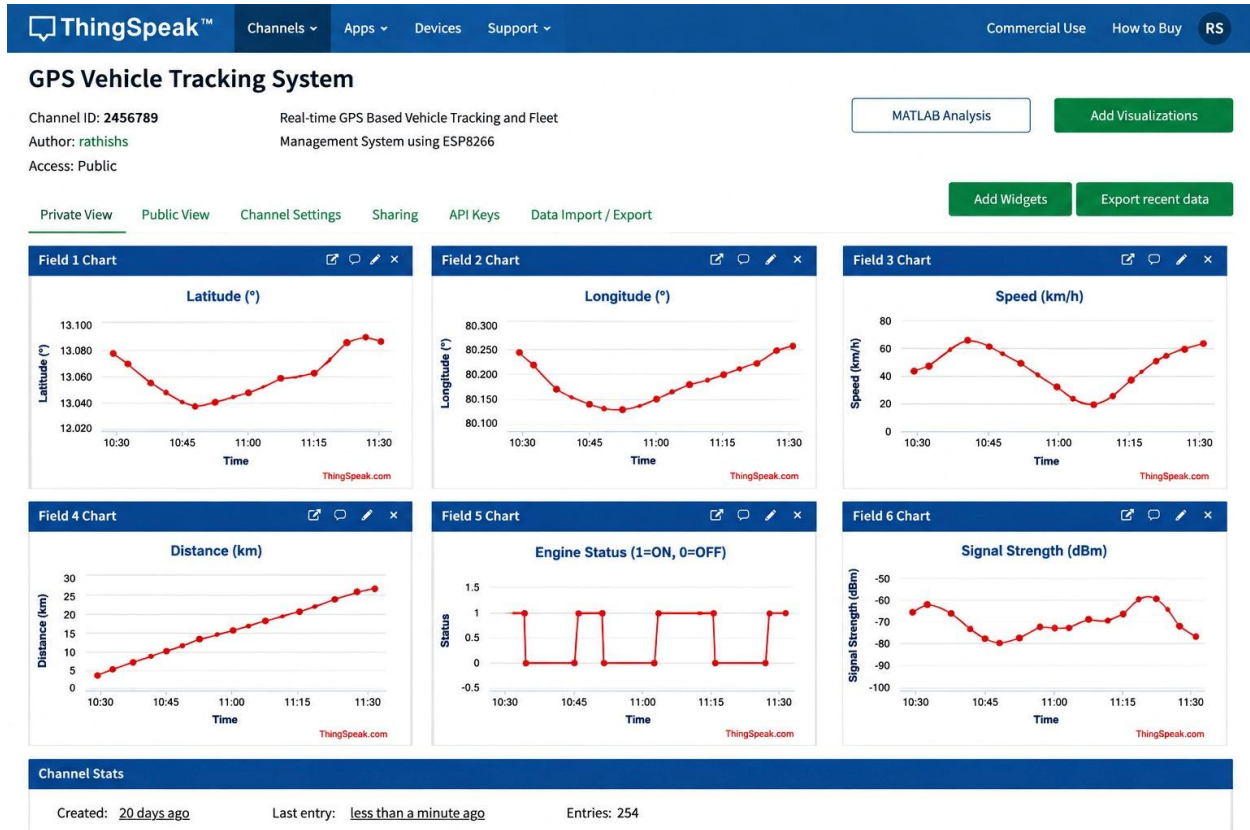
After identifying the color, the servo motor rotates to the corresponding position and directs the object to the appropriate section.

The ESP8266 simultaneously sends the sensor values and sorting information to ThingSpeak through Wi-Fi.



12. RESULTS & PERFORMANCE ANALYSIS

Expected Result



13.CONCLUSION

The IoT Based Automated Color Sorting System using ESP8266 provides a simple and economical solution for automatic color-based object sorting.

The combination of the IR sensor, TCS3200 color sensor, ESP8266, servo motor and ThingSpeak Cloud enables the system to detect, identify, sort and monitor objects automatically.

The project demonstrates how IoT and embedded technologies can be integrated with automation to reduce human effort, improve sorting efficiency and provide real-time monitoring of system performance.

The system can be further improved by adding more color sensors, a conveyor belt, additional sorting bins, LCD display, mobile notifications and advanced image-processing techniques.